



โจทย์ ปฏิบัติการรวบแพคเกจพัสดุ: นาที่วิกฤตบนสายพานก่อนรถออก (Operation Left-Pack)

สัญญาณเตือนปล่อยรถขนส่งดังขึ้นแล้ว! พัสดุทั้งหมดบนสายพานลำเลียงถูกจัดเรียงตามลำดับการขนส่งขึ้นรถไว้ล่วงหน้า แต่เนื่องจากเจ้าหน้าที่ตรวจสอบความปลอดภัยเพิ่งถึงพัสดุบางชิ้นออกไปตรวจจะทันหัน จึงเกิด "ช่องว่าง" แทรกคั่นอยู่ระหว่างพัสดุที่เหลือเป็นระยะ

เพื่อไม่ให้เกิดการขนถ่ายล่าช้า หัวหน้าคลังสินค้าจึงสั่งการให้ หุ่นยนต์จัดการสายพาน (Robo-Packer) เข้าไปรวบพัสดุเฉพาะใน "ช่วงพิภักดิ์ที่กำหนด" ให้เลื่อนมาเรียงชิดติดกันทางด้านซ้าย เพื่อส่งต่อให้พนักงานยกขึ้นรถได้อย่างต่อเนื่อง โดยมีกฎเหล็กสำคัญคือ หุ่นยนต์ต้องรักษาลำดับก่อน-หลังของพัสดุตามเดิมทุกประการ และห้ามแตะต้องพัสดุหรือช่องว่างที่อยู่นอกเขตพิภักดิ์ที่ได้รับมอบหมายโดยเด็ดขาด

สายพานลำเลียงนี้มีทั้งหมด N ช่อง เรียงจากซ้ายไปขวาเป็นตำแหน่งที่ 0 ถึง $N - 1$ แต่ละช่องบรรจุจำนวนเต็มหนึ่งค่า โดยเลข 0 แทนช่องว่างที่ไม่มีพัสดุ และจำนวนเต็มบวกแทนรหัสปลายทางของพัสดุชิ้นนั้น (พัสดุหลายชิ้นสามารถมีรหัสปลายทางซ้ำกันได้)

งานที่ต้องทำ:

จัดพัสดุตั้งแต่ตำแหน่ง L ถึง R โดยนับรวมทั้งสองตำแหน่ง ให้เป็นไปตามเงื่อนไขต่อไปนี้

- พัสดุทุกชิ้นในช่วงต้องอยู่ชิดด้านซ้ายของช่วง โดยไม่มีช่องว่างคั่น
- พัสดุที่อยู่ก่อนอีกชิ้นหนึ่งในช่วงเดิม ต้องยังอยู่ก่อนชิ้นนั้นหลังจัดเสร็จ ห้ามเรียงตามค่ารหัสหรือตัดพัสดुरหัสซ้ำกัน
- ช่องที่เหลือด้านขวาภายในช่วงต้องเป็น 0 และทุกช่องนอกช่วงต้องมีค่าเท่าเดิมเมื่อจัดเสร็จ ให้รายงานจำนวนพัสดุในช่วงที่จัด และสภาพสายพานทั้งเส้น

ฟังก์ชันที่ต้องเขียน:

จงเติมการทำงานของฟังก์ชันต่อไปนี้ในไฟล์ `parcels.cpp`

```
int pack_parcels(int* first, int count);
```

พารามิเตอร์	ความหมาย
first	Pointer ที่ชี้ไปยังช่องซ้ายสุดของช่วงสายพานที่ต้องจัด
count	จำนวนช่องทั้งหมดในช่วงที่ต้องจัด (นับรวมช่องที่ first ชี้อยู่ด้วย)

เงื่อนไขการทำงานของฟังก์ชัน:

1. การแก้ไขและคืนค่า: ฟังก์ชันต้องแก้ไขข้อมูลต้นฉบับผ่าน `first` โดยตรง และ `return` จำนวนพัสดุในช่วงนั้นกลับไป (หากไม่มีพัสดุเลยให้คืนค่า `0`)
2. การรับประกันจากระบบ: `first` จะไม่เป็น `nullptr` และชี้ไปยังสมาชิกของอาร์เรย์ `int` ที่อ่านและเขียนได้ต่อเนื่องอย่างน้อย `count` ตัวเสมอ
3. วิธีที่ระบบตรวจเรียกใช้: ระบบตรวจจะเรียก `pack_parcels(a + L, R - L + 1)` ดังนั้น `first` จึงชี้ที่ต้นช่วงอยู่แล้ว ทั้งนี้ตัวแปร `N`, `L` และ `R` เป็นข้อมูลภายในของระบบตรวจ นักเรียนไม่สามารถเรียกใช้ตัวแปรดังกล่าวในฟังก์ชันได้โดยตรง

ข้อมูลนำเข้า

ข้อมูลส่วนนี้รับโดย `main.cpp` ที่ให้มา นักเรียนไม่ต้องเขียนคำสั่งรับข้อมูลในฟังก์ชัน

- บรรทัดที่ 1 จำนวนเต็ม `N` คือจำนวนช่องทั้งหมดบนสายพาน
- บรรทัดที่ 2 จำนวนเต็ม `N` จำนวน คือค่าของช่องต่าง ๆ จากซ้ายไปขวา
- บรรทัดที่ 3 จำนวนเต็ม `L R` คือขอบซ้ายและขอบขวาของช่วงที่ต้องจัด ใช้ตำแหน่งเริ่มจาก `0` และนับรวมปลายทั้งสองด้าน

ข้อมูลส่งออก

- บรรทัดที่ 1 จำนวนเต็ม `k` คือจำนวนพัสดุในช่วงที่จัด ซึ่งเป็นค่าที่ฟังก์ชันคืนให้ระบบตรวจ
 - บรรทัดที่ 2 จำนวนเต็ม `N` จำนวน คือค่าสายพานทั้งเส้นหลังจัดเสร็จ คืนด้วยช่องว่าง
- ระบบตรวจเป็นผู้แสดงผลทั้งสองบรรทัด โดยอ่านค่าที่ฟังก์ชันคืนและข้อมูลในอาร์เรย์หลังเรียกฟังก์ชัน

ข้อจำกัด

- $1 \leq N \leq 200000$
- $0 \leq L \leq R < N$ จึงไม่มีช่องว่าง และ $1 \leq \text{count} \leq 200000$
- ค่าของแต่ละช่องอยู่ระหว่าง `0` ถึง `1000000000`
- เวลาทำงานของโปรแกรมไม่เกิน 2 วินาที และหน่วยความจำรวมไม่เกิน 256 MB

กติกาการเขียนและการส่ง

1. การเข้าถึงข้อมูลด้วย `Pointer`: ต้องอ่านและแก้ไขค่าของสมาชิกในสายพานผ่านการ Dereference ด้วยเครื่องหมาย `*` เท่านั้น (เช่น `*p` หรือ `*(first + i)`) ห้ามใช้เครื่องหมาย `[]` ในโค้ดส่วนที่นักเรียนเขียนโดยเด็ดขาด (เช่น ห้ามเขียน `first[i]`, `p[i]` หรือเปลี่ยนพารามิเตอร์เป็น `int first[]`) เนื่องจากข้อสอบนี้ต้องการวัดทักษะการใช้งาน `Pointer` โดยเฉพาะ

2. ตัวแปรและคำสั่งที่อนุญาต/ห้ามใช้: อนุญาตให้ประกาศเฉพาะตัวแปร `Pointer` ตัวแปรนับจำนวน และตัวแปรชั่วคราวพื้นฐานในจำนวนคงที่เท่านั้น ห้าม `#include` ไลบรารีอื่นเพิ่มเติม (นอกเหนือจาก `"parcels.h"` ที่มีให้) ห้ามสร้างอาร์เรย์หรือ Container เพิ่ม ห้ามจองหน่วยความจำแบบไดนามิก (`new`, `malloc`) และห้ามใช้ฟังก์ชันสำเร็จรูปในการจัดเรียงหรือย้ายข้อมูล (เช่น `std::sort`, `std::remove` หรือ `std::stable_partition`)

3. ประสิทธิภาพของอัลกอริทึม: ต้องออกแบบให้ใช้เวลาทำงานแปรผันตรงตามจำนวนช่องในช่วง หรือ $O(\text{count})$ (สามารถวนลูปผ่านช่วงข้อมูล 2 รอบได้) และใช้หน่วยความจำเสริมเป็นค่าคงที่ หรือ $O(1)$ กล่าวคือ จำนวนตัวแปรที่สร้างขึ้นต้องไม่เพิ่มขึ้นตามขนาดของข้อมูล

4. ขอบเขตและความเป็นอิสระของฟังก์ชัน: ฟังก์ชันต้องเข้าถึงข้อมูลเฉพาะภายในช่วง `count` ช่องที่กำหนดเท่านั้น (ห้ามอ่านหรือเขียนข้อมูลนอกช่วง) และต้องทำงานได้อย่างถูกต้องทุกครั้งที่ถูกเรียกใช้ โดยไม่อาศัยค่าที่ค้างอยู่จากการเรียกครั้งก่อน (ห้ามใช้ตัวแปร Global หรือ static เก็บสถานะค้างไว้)

5. รูปแบบไฟล์ที่ส่ง: ส่งเฉพาะไฟล์ `parcels.cpp` โดยต้องคงหัวฟังก์ชัน `int pack_parcels(int* first, int count)` ไว้ตามต้นฉบับ ห้ามเขียนฟังก์ชัน `main()` และห้ามใส่คำสั่งรับหรือแสดงผลข้อมูลใดๆ ในไฟล์ที่ส่ง

กติกาการให้คะแนน

- คะแนนเต็ม 50 คะแนน ตรวจสอบด้วยระบบ CMS Grader จำนวน 5 ชุด ชุดละ 10 คะแนน โดยแต่ละชุดต้องได้ผลลัพธ์ถูกต้องครบถ้วนทั้งค่าจำนวนพัสดุที่ฟังก์ชันคืนกลับ และข้อมูลในสายพานหลังจัดเสร็จ
- เงื่อนไขการได้ 0 คะแนนในข้อนี้ทันที (แม้ผลลัพธ์จากระบบ Grader จะถูกต้อง):
 - ปรากฏการใช้เครื่องหมาย `[]` ในโค้ดส่วนใดส่วนหนึ่งของไฟล์ `parcels.cpp` (เช่น เขียน `first[i]`, `p[i]` หรือแก้หัวฟังก์ชันเป็น `int first[]`)
 - ไม่มีการใช้ตัวดำเนินการ `*` (Dereference) ร่วมกับ Pointer ในการอ่านและเขียนข้อมูลลงในช่วงที่กำหนด
 - มีการ `#include` เฮดเดอร์ไฟล์อื่นเพิ่มเติมนอกเหนือจาก `"parcels.h"` หรือมีการสร้างอาร์เรย์/Container เพิ่ม จองหน่วยความจำเพิ่ม หรือเรียกใช้ฟังก์ชันสำเร็จรูปช่วยจัดข้อมูล

ตัวอย่างข้อมูลนำเข้าและข้อมูลส่งออก

ตัวอย่างที่ 1

ข้อมูลนำเข้า	ข้อมูลส่งออก
7 0 12 0 7 12 0 9 0 6	4 12 7 12 9 0 0 0

จัดทั้งสายพาน มีพัสดุ 4 ชิ้น รหัส 12 ทั้งสองชิ้นต้องอยู่ครบ และยังคงลำดับ 12, 7, 12, 9

ตัวอย่างที่ 2

ข้อมูลนำเข้า	ข้อมูลส่งออก
7 90 0 4 0 6 0 80 1 5	2 90 4 6 0 0 0 80

จัดเฉพาะตำแหน่ง 1 ถึง 5 พัดรหิส 90 และ 80 อยู่นอกช่วง จึงต้องอยู่ตำแหน่งเดิม

ตัวอย่างที่ 3

ข้อมูลนำเข้า	ข้อมูลส่งออก
4 0 0 0 0 0 3	0 0 0 0 0

ทุกช่องในช่วงว่างอยู่แล้ว จึงคืนจำนวนพัดรเป็น 0 และค่าสายพานไม่เปลี่ยน

ตัวอย่างที่ 4

ข้อมูลนำเข้า	ข้อมูลส่งออก
5 7 0 9 0 3 2 2	1 7 0 9 0 3

ช่วงที่จัดมีเพียงตำแหน่ง 2 ซึ่งมีพัดรหนึ่งชิ้น ช่องว่างนอกช่วงต้องคงเดิม

ตัวอย่างที่ 5

ข้อมูลนำเข้า	ข้อมูลส่งออก
4 6 2 6 1 0 3	4 6 2 6 1

ไม่มีช่องว่างในช่วง จึงคงข้อมูลเดิมทั้งหมด แม้มีรหัสซ้ำและรหัสไม่ได้เรียงจากน้อยไปมาก